

Microservice-based Application Development

Nazih Errahel

erraheln@ex.istu.edu

OUTLINE

- An example Application
- Monolithic Architecture
- Microservice Architecture

What will you learn ?

a basic understanding of microservices, containers, and cloud-native applications



SmartBuilding Application – An example

Features

- Add/Remove new smart devices/appliances, e.g. Coffee Machine, Refrigerator
- Energy consumption (daily, monthly consumption)
- Usage/Activities of appliances
- Alerts / Notification

Underline technologies...

- Database
- Frontend
- Prog. Language, many more

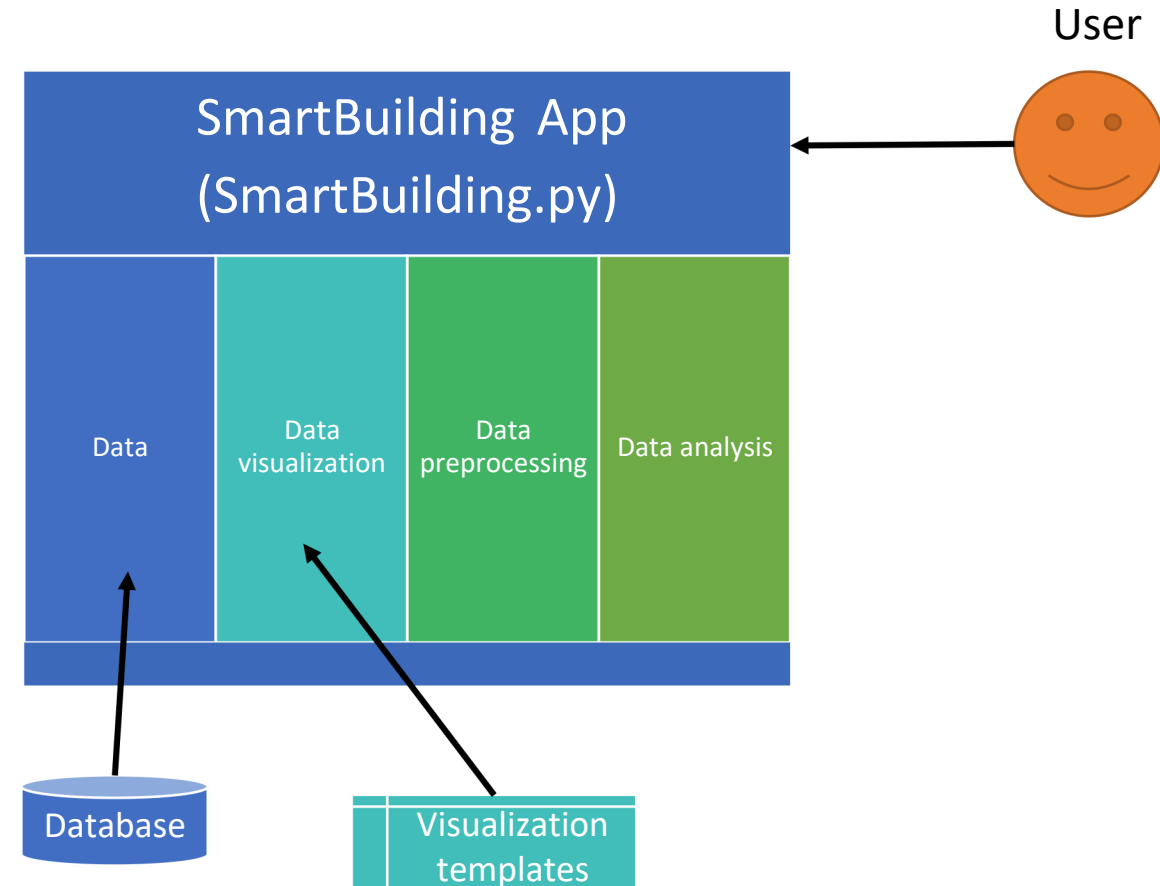
SmartBuilding Application - Monolithic approach

Example:

SmartBuilding.py - IoTdata

processing

- *read()....* : Read data (from multiple .csv files, from local file system)
- *outlier()*, *missing()*, ... : Preprocess the data (e.g. outlier, missing data)
- *mean()*, *max()* ...: Analyze the data
- *pushToMongo()*, *pushToS3()* ...: Push data to other database
- *viz()* ...: Visualize the data

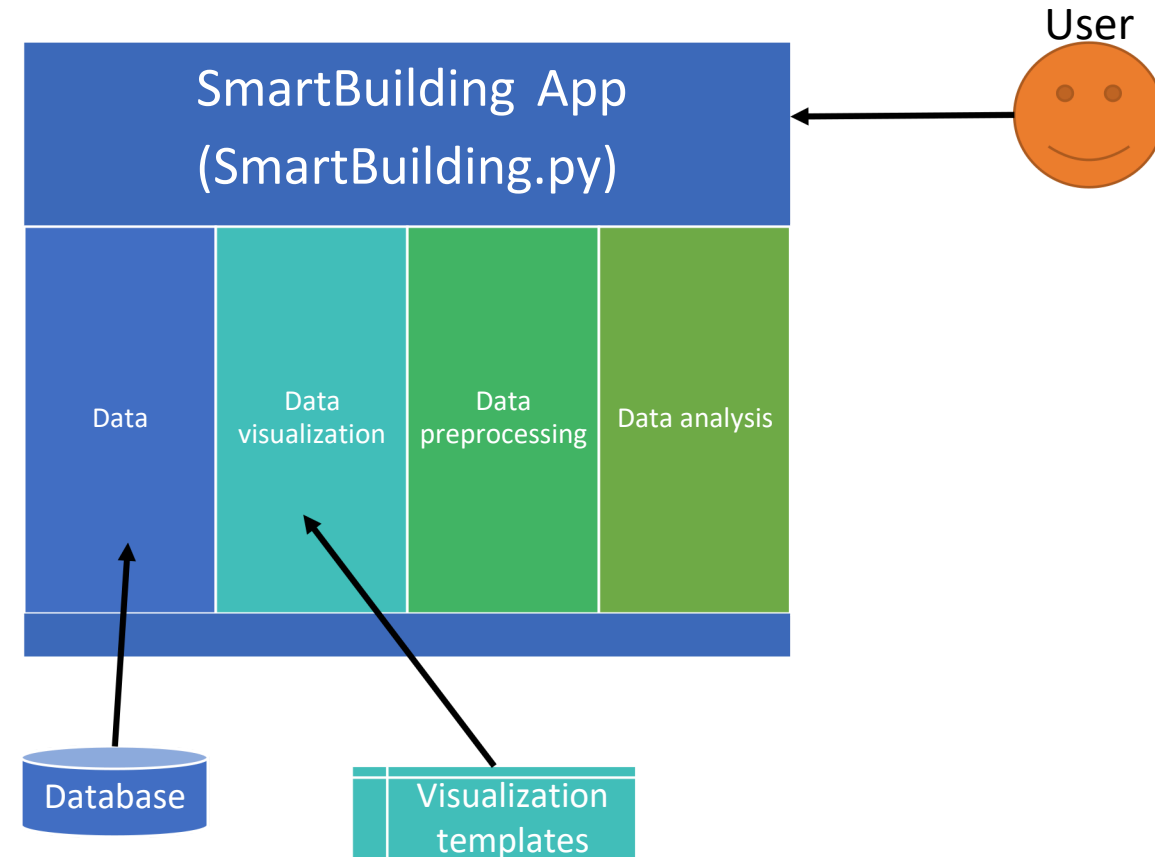


SmartBuilding Application - Monolithic approach

Example: *Decomposing monolith application*

SmartBuilding.py - IoTdata processing

- *read.py*-Read data (from multiple .csv files)
- *outlier.py* , *missing.py* ...- Preprocess the data (e.g. outlier, missing data)
- *mean.py*, *max.py*...-Analyze the data
- *pushToMongo.py*, *pushToS3.py*...- Push data to other database
- *viz.py*- Visualize the data

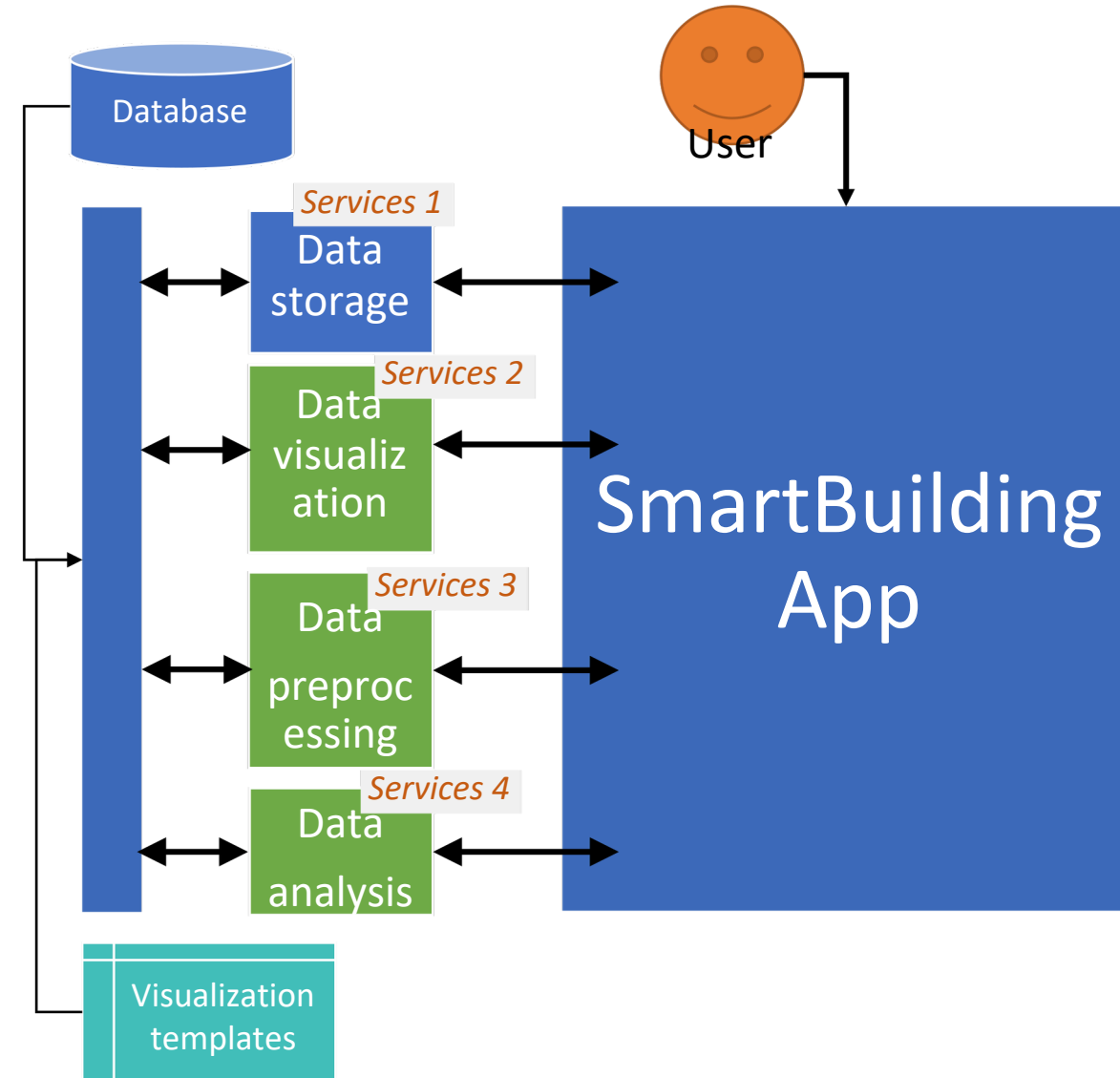


SmartBuilding Application - Microservice Approach

Example: *Decomposing monolith application*

SmartBuilding App - IoT data processing

- *ConsumeData Service*- Read data (from multiple .csv files) (*read.py*)
- *Outlier detection service, Missing data service*-Preprocess the data (e.g. outlier, missing data) (*outlier.py, missing.py*)
- *Data analytics service* -Analyze the data (*mean.py, max.py*)
- *Export data service*...-Push data to other database (*pushToMongo.py, pushToS3.py*)
- *Visualization service* - Visualize the data (*viz.py*)



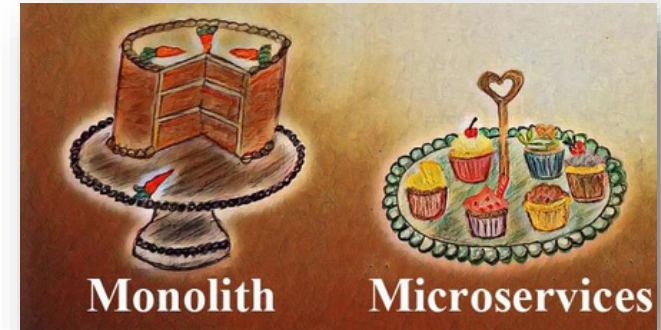
Monolith vs Microservice – in general

Monolith vs Microservice

Monolithic vs microservices



Src: https://devcamp.com/site_blogs/monolith-vs-microservice-rails-applications



Src: <https://www.linkedin.com/pulse/monolith-vs-microservices-serverless-why-should-business-gorini/>

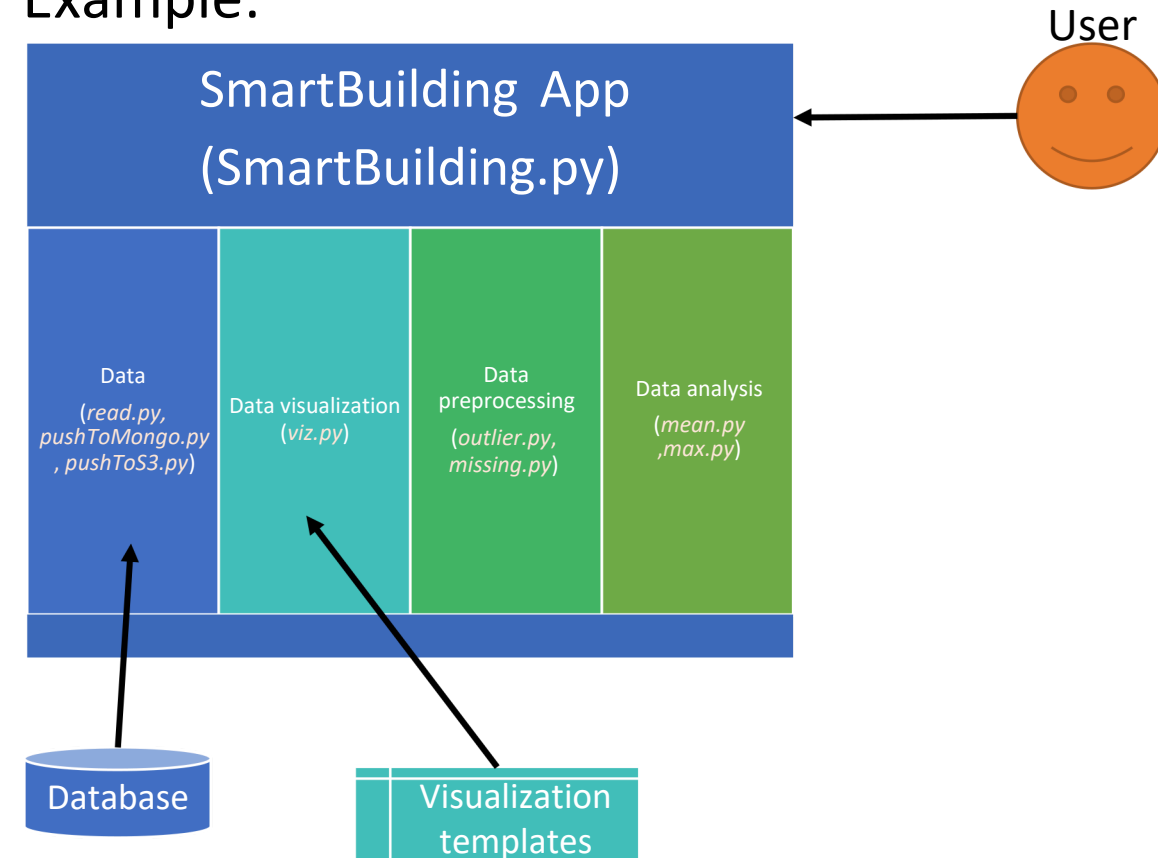


Src: <https://blog.knoldus.com/why-do-you-always-choose-microservices-over-me-said-the-monolithic-architecture/>

Monolithic Approach

- *Very tightly coupled*
 - *Interdependent modules*
 - *Change in module requires a update in other module*
- *Usually has three blocks:*
 - *Database*
 - *Server-side application*
 - *Client UI*
- *Usually this is a centralized approach*

Example:

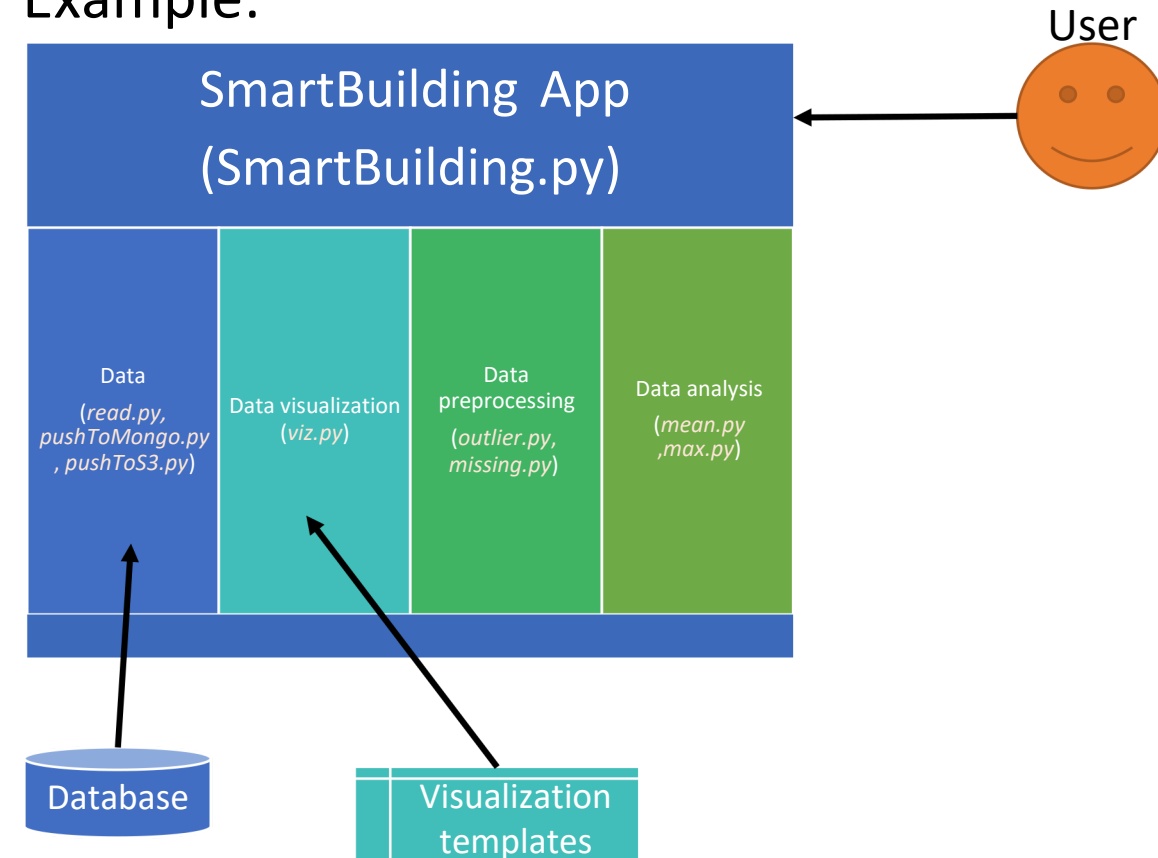


** Only applicable to very small size application*

Monolithic Approach -benefits

- *Development**
 - *Very few python files to develop*
- *Simplified Testing**
- *Deployment**
 - *Simply pack the pythons and deploy*
- *Faster Scale**
 - *Scale up / Vertical scaling*
 - *Scale out / Horizontal Scaling*

Example:

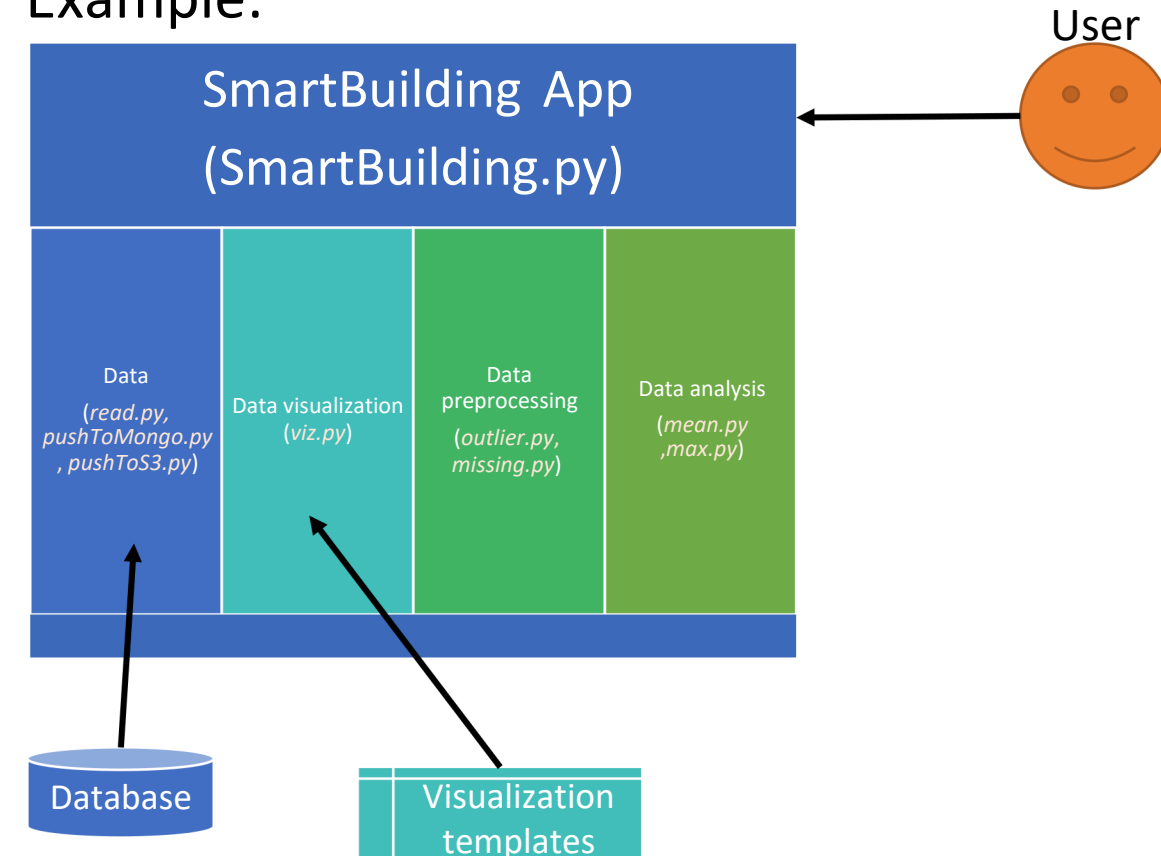


** Only applicable to very simple, small size application*

Monolithic Approach -limitations

- *Not suitable for **large applications***
- *Slow application development*
 - Difficult to understand the structure
 - The IDEs may not work fine with large codebase
 - E.g. Can you edit 15MB PDF smoothly?
- *Difficult to scale*
 - E.g. what will happen to *PushToMongo.py* when the data size increases to GBs
- *Reliability*
 - Due to higher interdependency
 - E.g. Minor bug in *read.py* may crash whole system
- *Flexibility of underlined software stack*
 - Can we develop each module using different technologies?

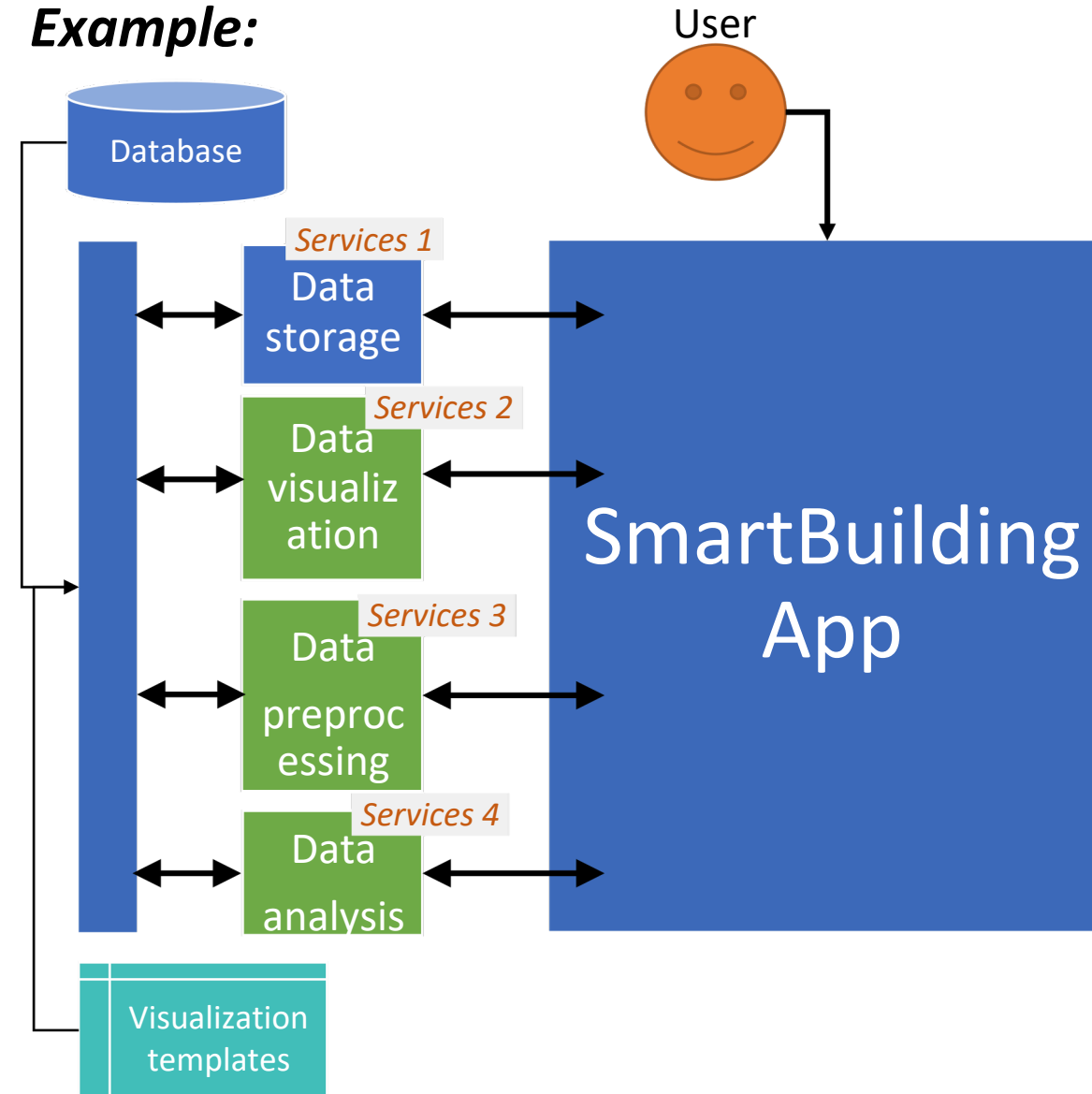
Example:



Microservice Approach

- Collection of *smaller* and *independent* services
- Usually this is a *distributed* approach
- **GOAL**: to accelerate software development by enabling continuous delivery/deployment.
- Services communicate via **APIs**
- Service:
 - Dedicated function
 - Codebase
 - Processes Lifecycle
 - Database

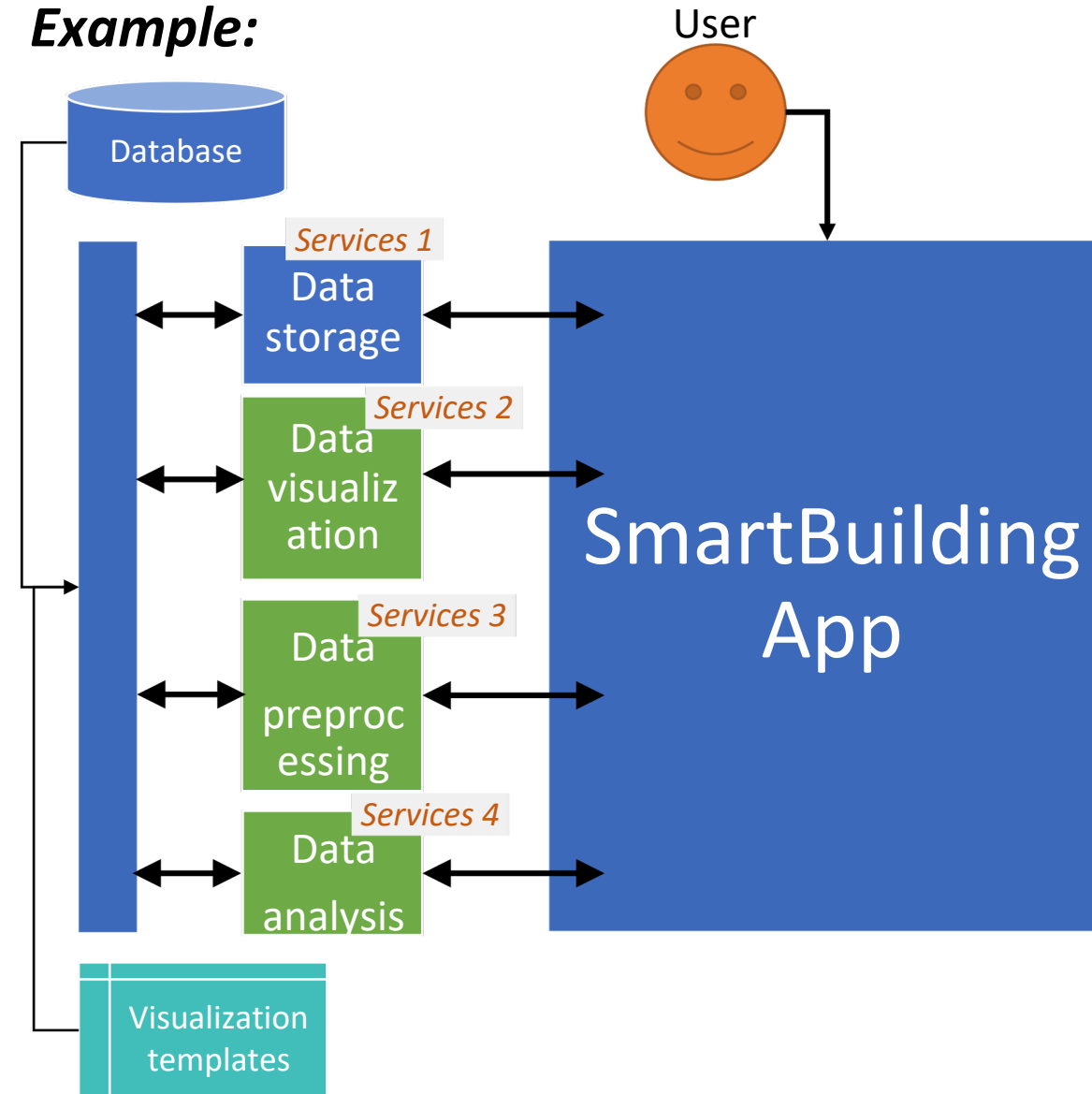
Example:



Microservice Approach

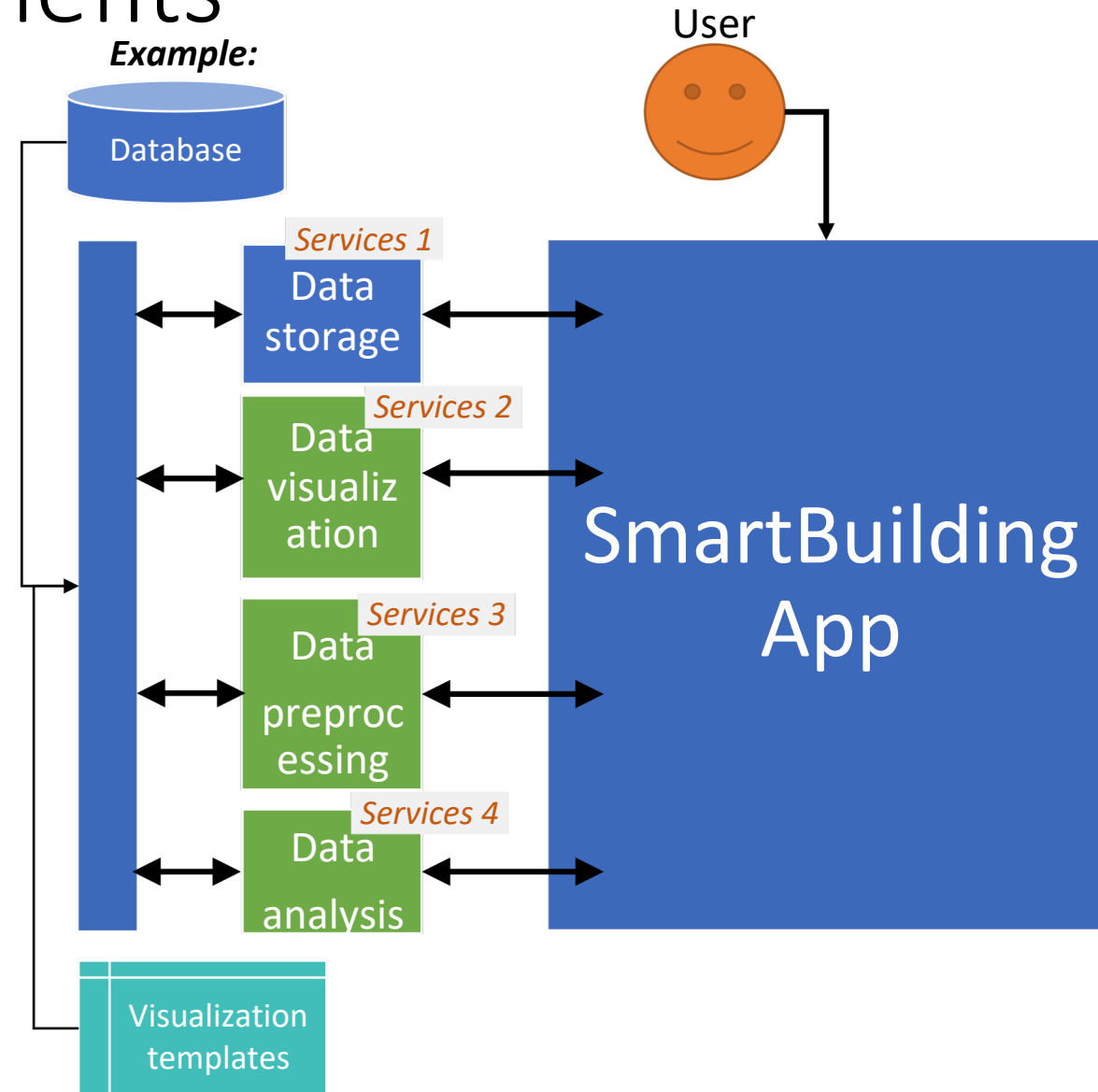
- Service Lifecycle
 - Created
 - Installed
 - Configured
 - Start
 - Stop
 - Delete/remove
- Suitable for large team, large product/application

Example:



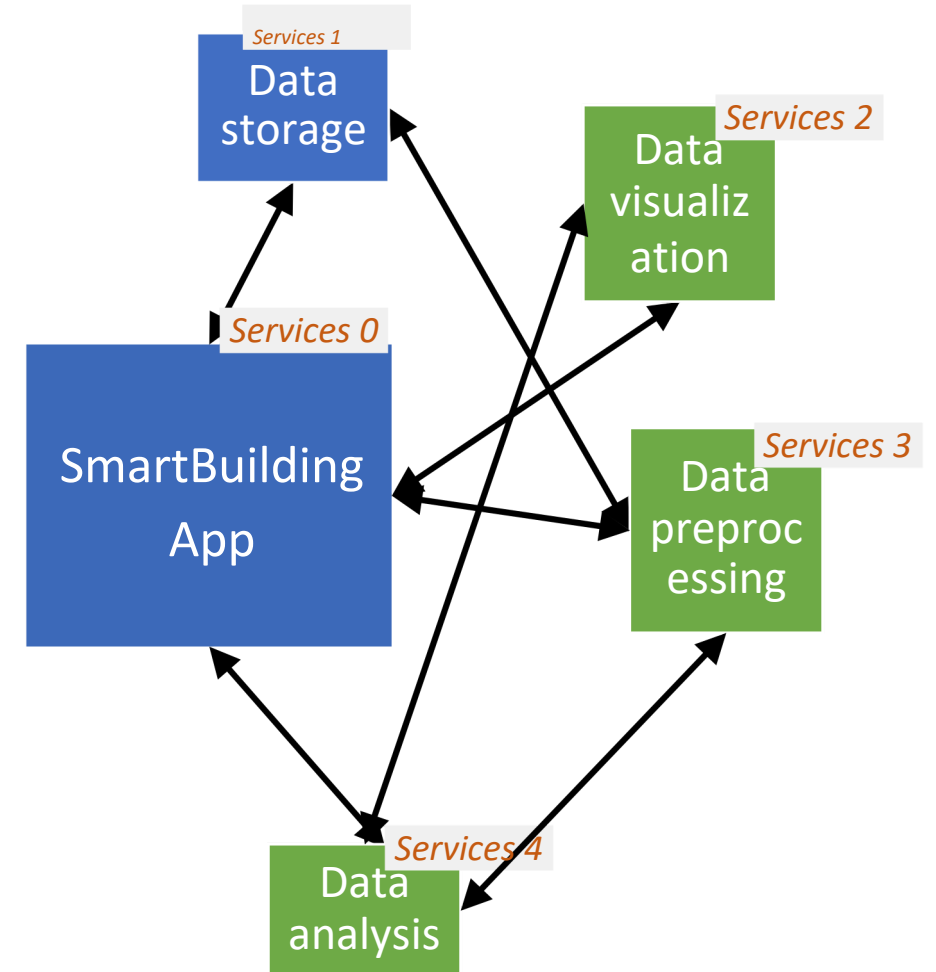
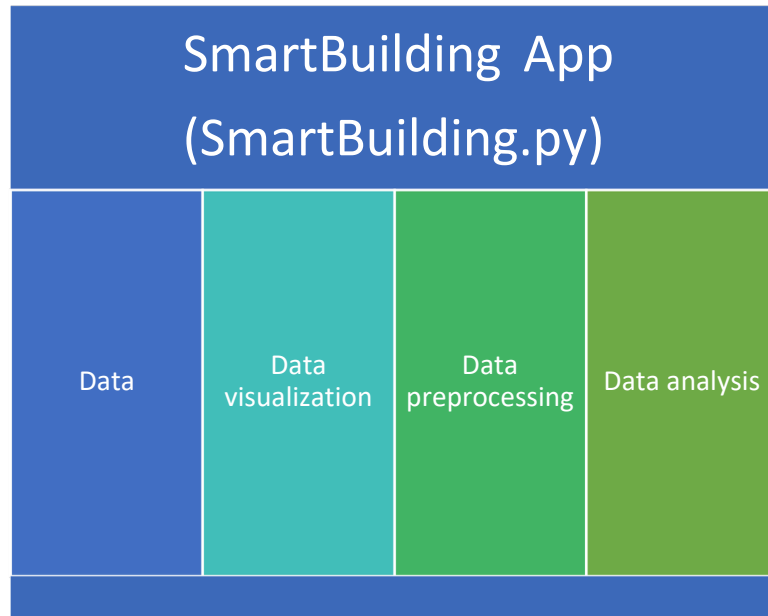
MicroserviceApproach –benefits

- Service:
 - Independent development
 - Independent test
 - Independent development
 - Independent update
- Scaling
 - E.g. create 5 replicas of data preprocessing service when the more frequent is arriving from the sensor
 - Able to scale Horizontally and vertically
- Improved fault isolation



Monolithic -> Microservice Architecture

Example:



So how the microservices communicate ?

- Communication Types:

- **Synchronous :**

- HTTP/HTTPS is a synchronous protocol.
 - The client sends a request and waits for a response from the service.
 - client code can only continue its task when it receives the HTTP/HTTPS server response

- **Asynchronous :**

- Sender usually doesn't wait for a response.
 - Protocols like AMQP (a protocol supported by many operating systems and cloud environments) uses asynchronous messages.
 - E.g. RabbitMQ, Kafka

- One-to-one Communication

- One-to-many Communication

So how the microservices communicate ?

- Communication Types:

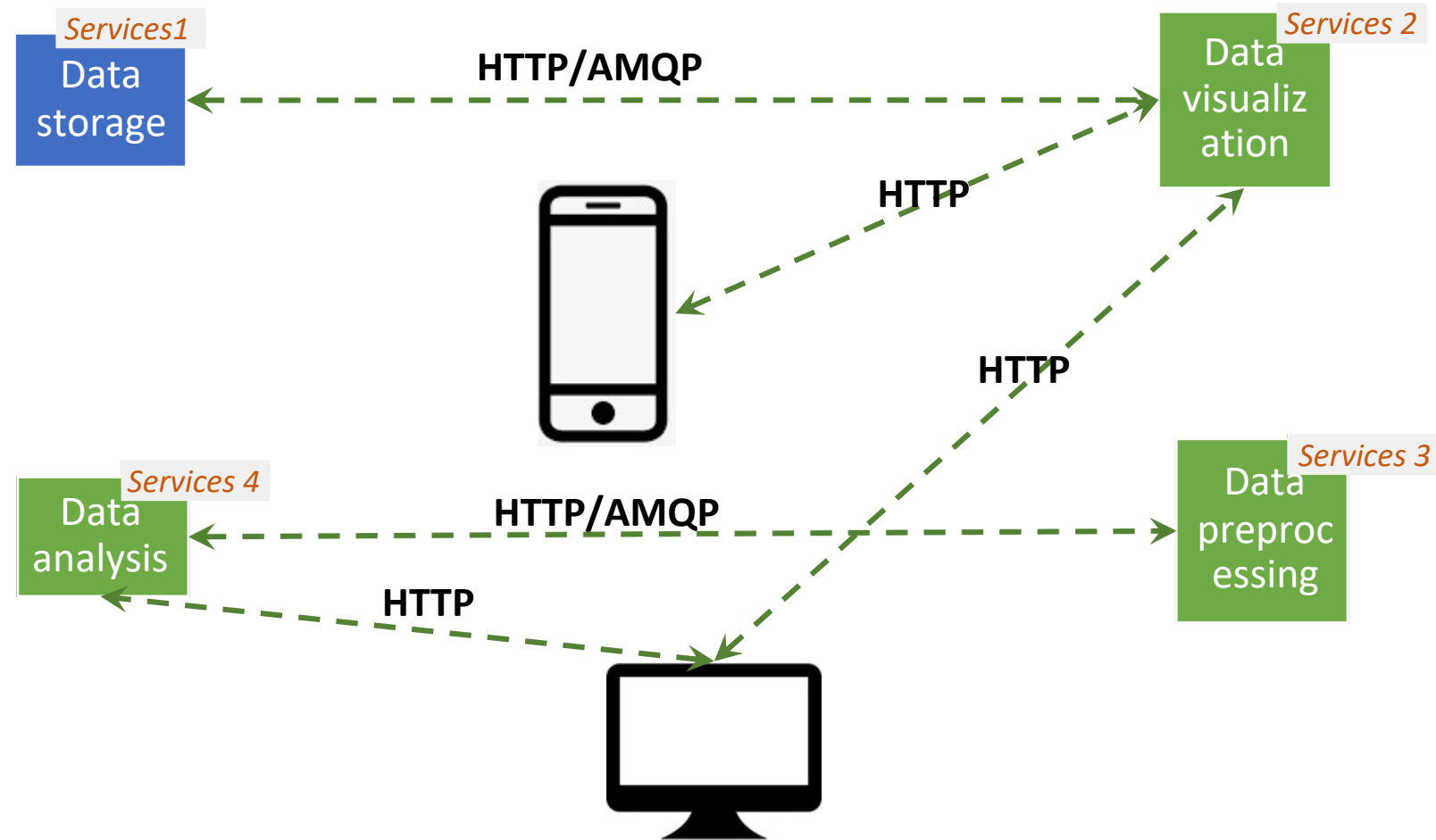
- One-to-one Communication

- A request must be processed by exactly one service (single producer and single receiver).
- E.g. Finding the min. temp of a particular date from the temp. sensor data
- Can be *synchronous communication*

- One-to-many Communication

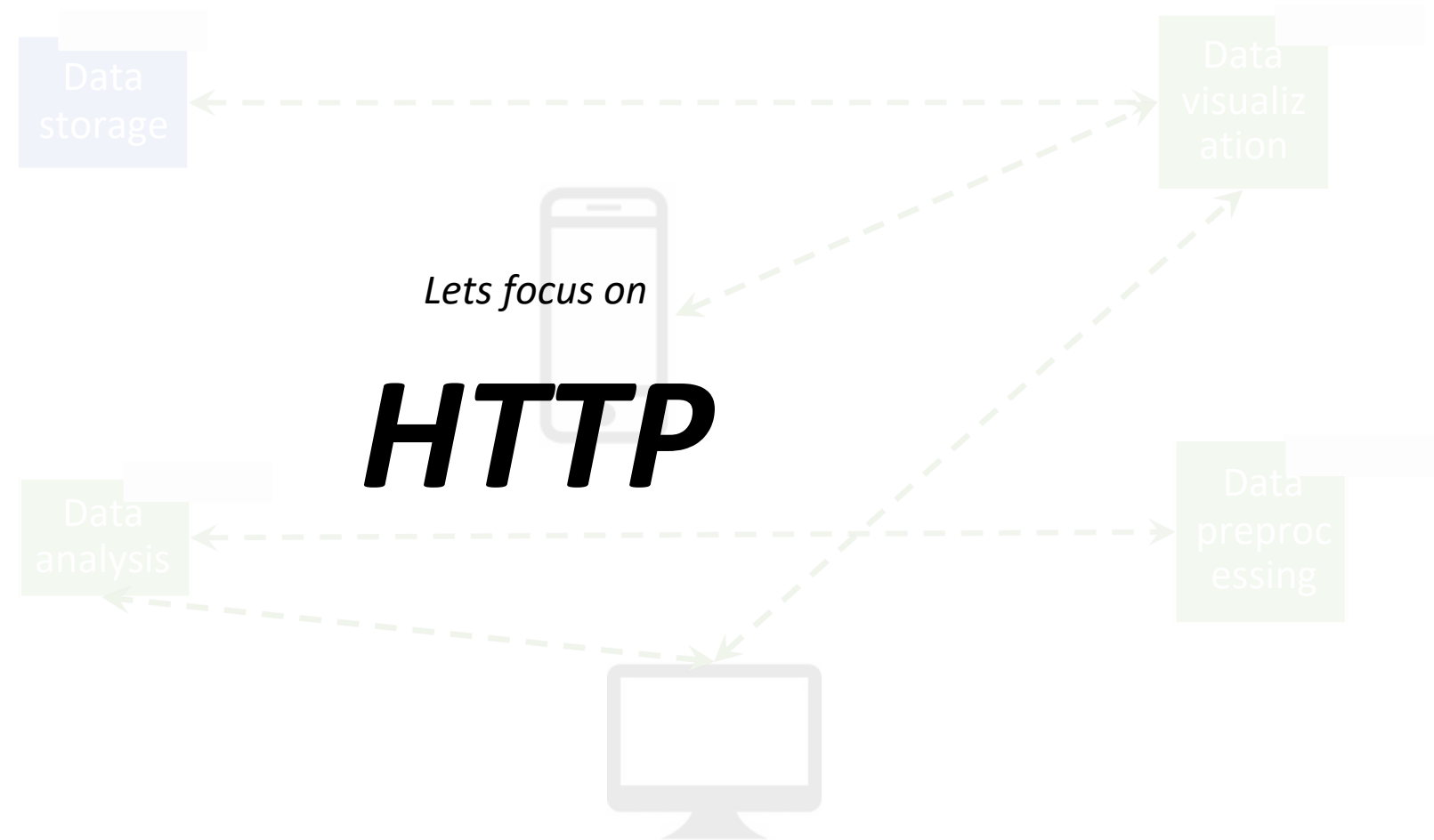
- Each request can be processed by zero to multiple receivers.
- Must be *asynchronous communication*
- Used in Event-driven architecture
- example is the publish/subscribe mechanism

So how the microservices communicate ?



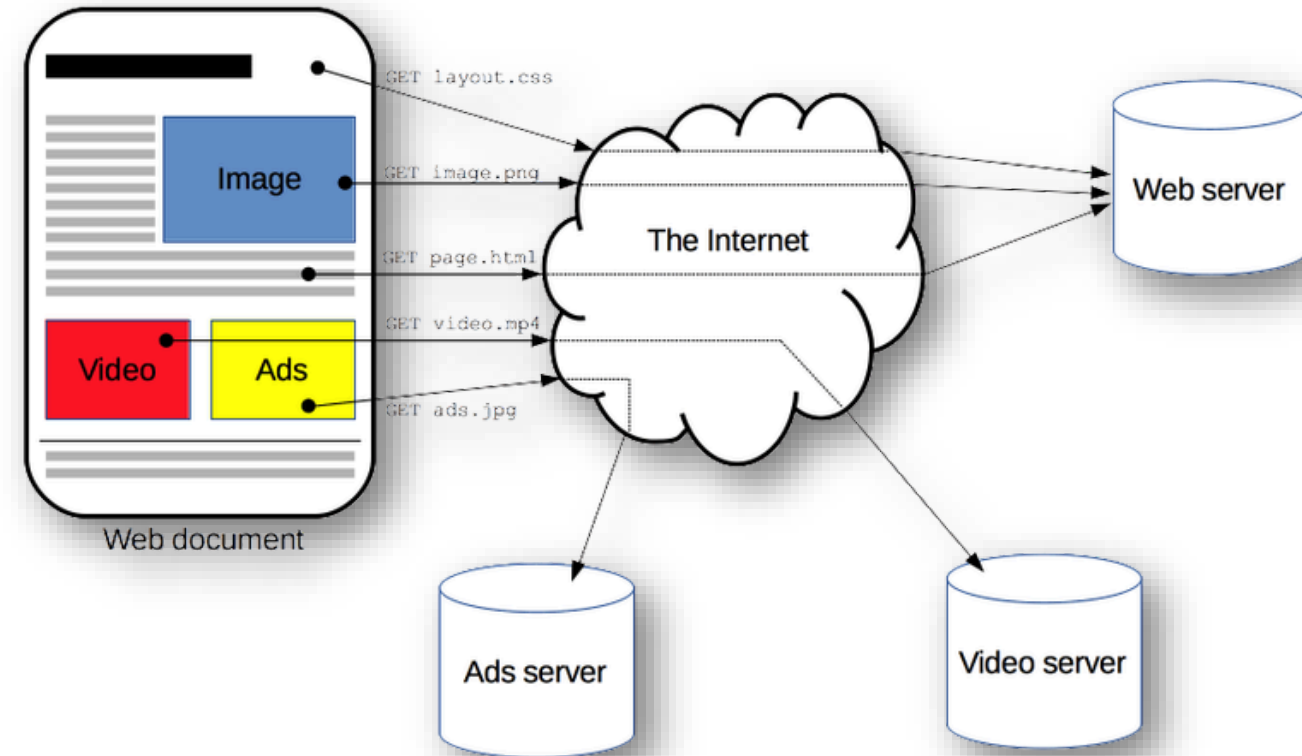
inter-process communication protocol such as HTTP, AMQP, or a binary protocol like TCP,

So how the microservices communicate ?



HTTP

- **HTTP** is a protocol for fetching resources such as HTML documents.
- Application layer (top layer) protocol for the World Wide Web
- foundation of any data exchange on the Web
- Follows the Request-Response model
- a client-server protocol,
 - Requests are initiated by the recipient, usually the Web browser.



Src: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>

HTTP Flow

When a client wants to communicate with a server :

- Client opens a TCP connection:



- Client Send an HTTP message:

```
GET / HTTP/1.1
Host: developer.mozilla.org
Accept-Language: fr
```



- Read the response sent by the server, such as:

```
HTTP/1.1 200 OK
Date: Sat, 09 Oct 2010 14:28:02 GMT
Server: Apache
Last-Modified: Tue, 01 Dec 2009 20:18:22 GMT
ETag: "51142bc1-7449-479b075b2891b"
Accept-Ranges: bytes
Content-Length: 29769
Content-Type: text/html
```

HTTP Request Methods

Some basic HTTP request methods are:

- GET
 - requests are presentation of the specified resource
 - should only be used to request data (they shouldn't include data).
- PUT
 - creates a new resource or replaces a representation of the target resource with the request payload.
- POST
 - sends data to the server
 - Posting a message to a bulletin board, newsgroup, mailing list, or similar group of articles
- DELETE
 - deletes the specified resource.

HTTP Status codes

- 200: OK success
- 201: Created
- 202: Accepted
- 400 : Bad Request (server could not understand the request due to invalid syntax)
- 404 : Not found
- 408: Request Timeout
- 500 : Internal Server Error
- 502 : Bad Gateway
- 503 : Service Unavailable

Complete list of status codes: <https://www.iana.org/assignments/http-status-codes/http-status-codes.xhtml#http-status-codes-1>

How do microservices send data to others?

- Message Format:
 - JSON
 - XML
 - BinaryFormat

How do microservices send data to others?

- Message Format:

- **JSON**

- XML

- Binary Format : Very selective usage

JSON file

- JSON: **J**ava**S**cript **O**bject **N**otation
- Lightweight format for storing and transporting data
 - language-independent data interchange format
- JSON is often used when data is sent from a server to a web page
- "self-describing" and easy to understand
- .json file extension
- Example:

```
{  
  "employees": [  
    {"firstName": "John", "lastName": "Doe"},  
    {"firstName": "Anna", "lastName": "Smith"},  
    {"firstName": "Peter", "lastName": "Jones"}  
  ]  
}
```

JSON file

- Curly braces hold objects
- Square brackets hold arrays
- Data is separated by commas
- Data is in name/value pairs
- *Example:*

```
{  
  "employees": [  
    {"firstName": "John", "lastName": "Doe"},  
    {"firstName": "Anna", "lastName": "Smith"},  
    {"firstName": "Peter", "lastName": "Jones"}  
  ]  
}
```

- *More info:* <https://www.json.org/json-en.html>

Presentation Topics:

1. **Event-Driven Architecture** (you may use kafka as an example)
2. **Microservice vs Serverless**

References

- <https://www.nginx.com/blog/refactoring-a-monolith-into-microservices/>
- <https://docs.microsoft.com/en-us/dotnet/architecture/microservices/architect-microservice-container-applications/communication-in-microservice-architecture>
- <https://solace.com/blog/microservices-advantages-and-disadvantages/>
- <https://docs.microsoft.com/en-us/dotnet/architecture/microservices/docker-application-development-process/docker-app-development-workflow>
- <https://www.bmc.com/blogs/containers-vs-microservices/>
- <https://swagger.io> <https://www.nginx.com/blog/building-microservices-using-an-api-gateway/>
- <https://developer.cisco.com/learning/lab/what-are-rest-apis/step/1>
- https://ois2.ttu.ee/uusois!/uus_ois2.ois_public.page_?_page=DF9175AB402E2383E097925418BCFB6818116956F635EC3A90AECEB2532C8A79&p_id=0A1A6F159CACB901